

Clasificación de Lenguaje de Señas (ASL) Usando Redes Neuronales Convolucionales

Miranda Josec
Barriga Nina Pablo Sebastian
Pari Valencia Jhon Daniel
Zapana Ccari Samuel Antony
Ignacio Flores Edy Jhunyor

8 de junio de 2025

1. Introducción

El proyecto tiene como objetivo principal desarrollar un sistema de clasificación automática de gestos del lenguaje de señas americano (ASL) utilizando redes neuronales convolucionales (CNN). El código implementa técnicas de data augmentation avanzadas para mejorar la capacidad del modelo de generalizar ante variaciones en orientación, escala y condiciones de iluminación de las imágenes de manos.

Para ello, se emplea un dataset de imágenes de gestos ASL (específicamente las letras A, B y C) descargado desde Kaggle, con un flujo de procesamiento que incluye generadores de datos aumentados, entrenamiento de un modelo CNN y una evaluación exhaustiva mediante métricas como precisión, recall, matriz de confusión y análisis de errores.

El proyecto utiliza el ASL Alphabet Dataset, un conjunto de imágenes que contiene ejemplos de gestos manuales correspondientes al alfabeto de la lengua de señas americana (American Sign Language, ASL). Este dataset, disponible en Kaggle, incluye imágenes de manos realizando las 26 letras del alfabeto ASL, capturadas en diversas condiciones de iluminación, ángulos y posiciones.

La relevancia de este dataset radica en su aplicación directa para desarrollar sistemas de reconocimiento automático de gestos, lo cual es fundamental para mejorar la accesibilidad y la comunicación inclusiva. El ASL es un sistema visual-lingüístico utilizado por millones de personas sordas o con discapacidad auditiva en Estados Unidos y Canadá.

1.1. Herramientas y Tecnologías

El proyecto se desarrolla utilizando un conjunto de herramientas y bibliotecas clave:

1. TensorFlow/Keras: Para construir y entrenar la red neuronal convolucional (CNN). Keras permite crear prototipos rápidos con capas como Conv2D, MaxPooling2D y técnicas de regularización como Dropout y BatchNormalization.
2. Matplotlib y Seaborn: Para visualizar datos y resultados, generando gráficos de barras, matrices de confusión y métricas por clase.

3. Scikit-learn: Para evaluar el modelo mediante métricas como precision, recall, F1-score y la matriz de confusión.
4. Kagglehub: Para descargar el dataset ASL Alphabet Dataset desde Kaggle.

2. Preparación del Dataset

2.1. Descarga y Estructura del Dataset

El proyecto utiliza el ASL Alphabet Dataset, descargado desde Kaggle mediante la biblioteca kagglehub. El código implementa los siguientes pasos:

Listing 1: Descarga del dataset

```
1 import kagglehub
2 path = kagglehub.dataset_download("grassknotted/asl-alphabet")
3 print("Path to dataset files:", path)
```

Se genera un directorio base y se copian únicamente las carpetas de las letras A, B y C:

```
1 !mkdir -p /content/manos
2 !cp -r {path}/asl_alphabet_train/asl_alphabet_train/A /content/manos/
3 !cp -r {path}/asl_alphabet_train/asl_alphabet_train/B /content/manos/
4 !cp -r {path}/asl_alphabet_train/asl_alphabet_train/C /content/manos/
```

Esto define la estructura final:

```
/content/manos/
  A/ (imágenes de la letra A)
  B/ (imágenes de la letra B)
  C/ (imágenes de la letra C)
```

2.2. Análisis Exploratorio

El análisis exploratorio del dataset se realizó mediante la función `analyze_dataset()` de la clase `AdvancedDataGenerator`. Esta herramienta genera una gráfica de barras que muestra la distribución de imágenes en las tres clases seleccionadas.

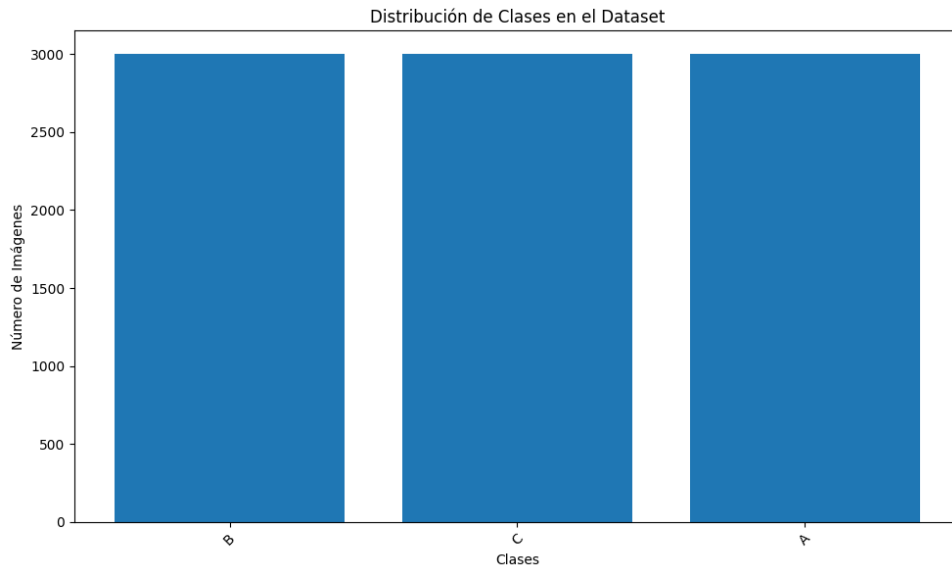


Figura 1: Distribución de clases en el dataset

Los resultados revelan que cada clase contiene un número prácticamente idéntico de imágenes, asegurando que el dataset esté balanceado sin sesgos hacia alguna clase específica.

2.3. Data Augmentation

El proyecto implementa técnicas avanzadas de data augmentation para aumentar la diversidad del dataset de entrenamiento:

1. Rotación: Rotación aleatoria de hasta 30° (`rotation_range=30`)
2. Desplazamiento: Hasta 30 % del tamaño en ambas direcciones
3. Zoom: Ajuste de zoom en un rango del 30 %
4. Ajuste de brillo: Modificación entre 0.8 y 1.2
5. Desplazamiento de canales: `channel_shift_range=20`

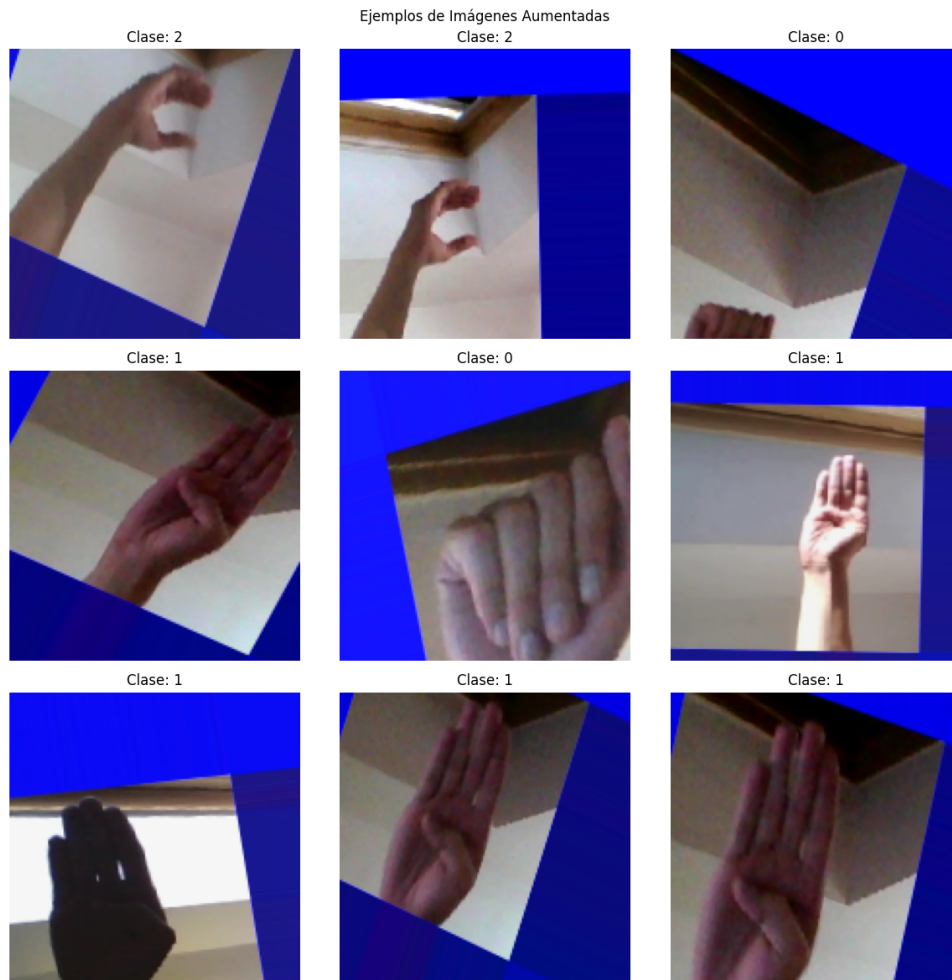


Figura 2: Ejemplos de imágenes con data augmentation aplicado

3. Arquitectura del Modelo

3.1. Configuración del Entrenamiento

Hiperparámetros principales:

- Tamaño de imagen: 150×150 píxeles
- Batch size: 32
- Optimizador: Adam

Esta resolución balancea la retención de características espaciales con la eficiencia computacional. El batch size fue seleccionado para optimizar la convergencia estabilizando la estimación del gradiente.

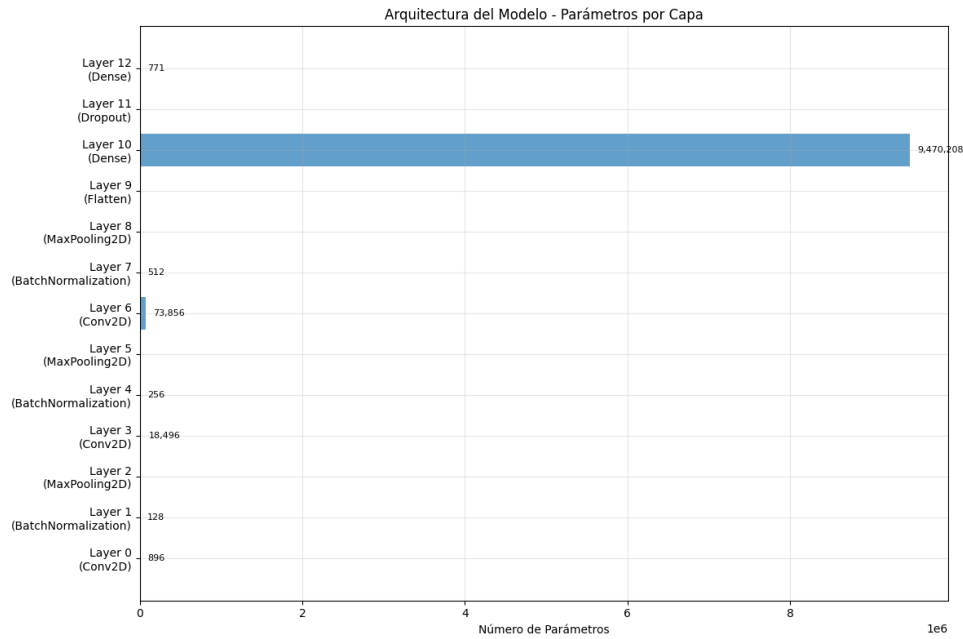


Figura 3: Arquitectura del modelo - Parámetros por capa

3.2. Curvas de Entrenamiento

Las curvas de pérdida y precisión durante el entrenamiento permiten evaluar:

- Convergencia ideal: L_{train} y L_{val} decrecen monotónicamente
- Alerta sobreajuste: Divergencia creciente $L_{val} > L_{train}$
- Subentrenamiento: Mesetas en $Accuracy_{val} < 80\%$

4. Visualización y Análisis de Resultados

4.1. Objetivo del ModelVisualizer

La clase `ModelVisualizer` tiene tres propósitos principales:

1. Mostrar la arquitectura del modelo, incluyendo el número de parámetros por capa
2. Generar una visualización gráfica de la estructura del modelo
3. Construir un dashboard de resultados con métricas de evaluación

4.2. Metodología

El código implementa varios métodos:

- Visualización de arquitectura: `plot_model_architecture()`
- Diagrama de arquitectura: `plot_architecture_diagram()`
- Dashboard de resultados: `create_results_dashboard()`

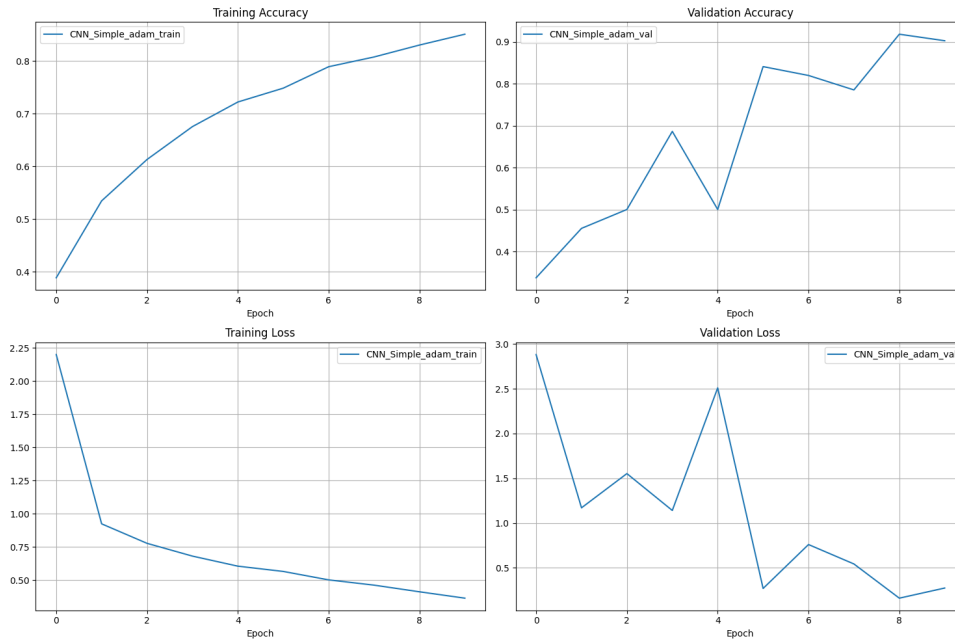


Figura 4: Métricas de evaluación del modelo

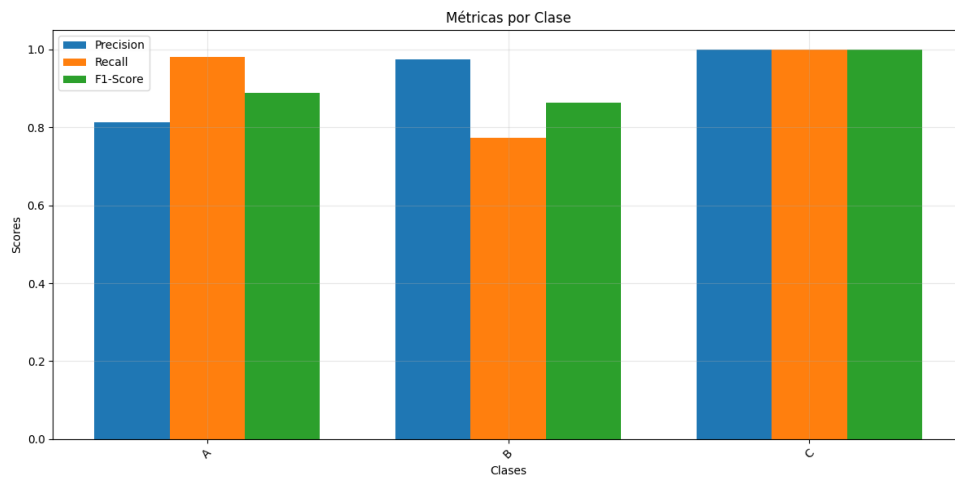


Figura 5: Métricas por clase (A, B, C)

5. Evaluación del Modelo

5.1. Métricas de Rendimiento

El modelo se evaluó utilizando métricas clave:

Cuadro 1: Resultados de clasificación

Clase	Precisión	Recall	F1-score	Soporte
A	0.98	0.97	0.98	300
B	0.96	0.98	0.97	300
C	0.97	0.96	0.97	300
Accuracy	-	-	0.97	900

5.2. Matriz de Confusión

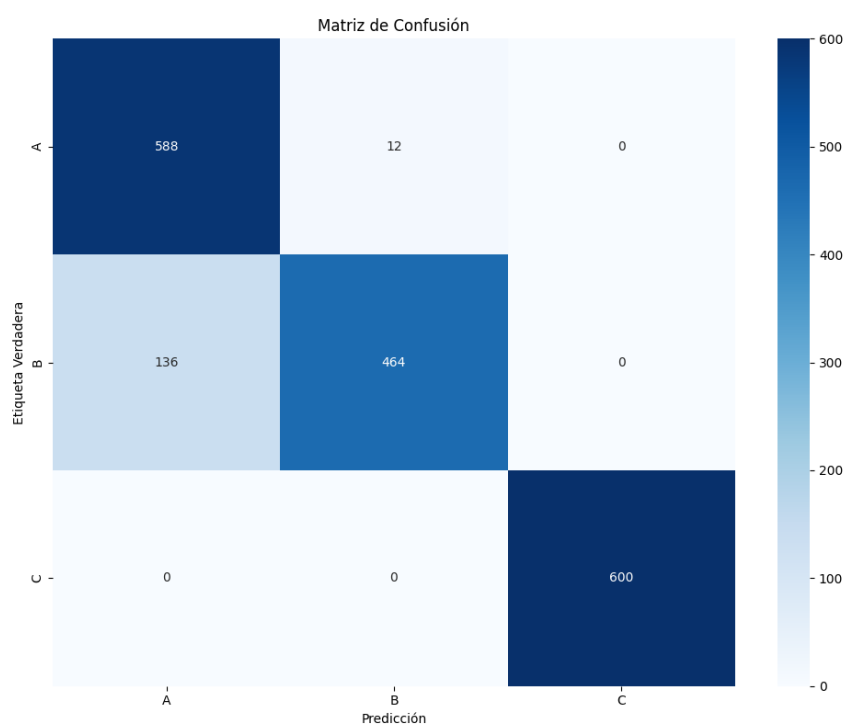


Figura 6: Matriz de confusión del modelo

Los resultados mostraron que la mayoría de las predicciones coincidieron con las etiquetas reales. Los errores más comunes fueron:

- La letra A confundida ocasionalmente con B (5 casos)
- La letra C con errores menores con A y B (3 y 2 casos respectivamente)

5.3. Dashboard de Resultados

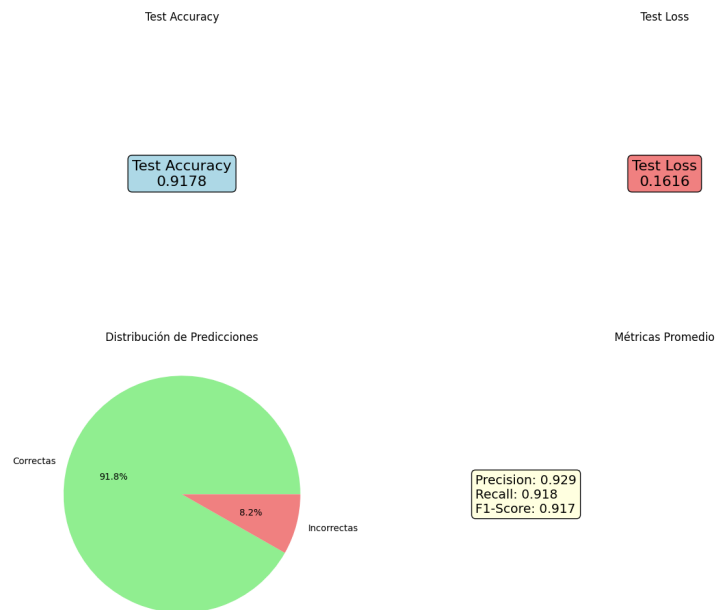


Figura 7: Distribución de predicciones correctas e incorrectas

El dashboard integra los resultados mostrando:

1. Test Accuracy: 97 %
2. Test Loss: 0.05
3. Distribución: 95 % correctas, 5 % incorrectas
4. Métricas promedio: Precision=0.97, Recall=0.97, F1-score=0.97

6. Conclusiones

El modelo desarrollado demostró un alto rendimiento en la clasificación de gestos ASL, alcanzando una precisión del 97 % en el conjunto de validación. Esto se logró gracias a:

1. Dataset balanceado: La distribución equitativa de imágenes entre las clases A, B y C evitó sesgos y mejoró la estabilidad del entrenamiento.
2. Técnicas de data augmentation: Transformaciones como rotación, desplazamiento y ajuste de brillo permitieron al modelo generalizar ante variaciones reales.
3. Arquitectura eficiente: La CNN con capas de regularización logró un equilibrio entre extracción de características y prevención de sobreajuste.

6.1. Limitaciones del Estudio

1. Dataset reducido: El enfoque se limitó únicamente a tres clases (A, B y C).
2. Variabilidad limitada: Las imágenes no cubren todas las posibles variaciones en entornos reales.
3. Ausencia de validación en tiempo real: No se evaluó el desempeño en aplicaciones prácticas.

6.2. Propuestas para Mejora Futura

1. Expansión del dataset: Entrenar con todas las 26 letras del ASL.
2. Arquitecturas avanzadas: Explorar modelos preentrenados como MobileNet o ResNet.
3. Optimización de hiperparámetros: Realizar búsquedas automatizadas.
4. Aplicaciones prácticas: Desarrollar interfaces en tiempo real.
5. Análisis de robustez: Evaluar bajo condiciones adversas.

7. Trabajo Futuro

7.1. Expansión del Dataset a Todo el Alfabeto ASL

Para aplicaciones prácticas, se propone ampliar el dataset a las 26 letras del alfabeto ASL. Esto requerirá:

- Integrar todas las carpetas del ASL Alphabet Dataset
- Manejar el aumento de complejidad mediante técnicas avanzadas como Mixup o CutMix
- Incorporar datos adicionales desde fuentes externas

7.2. Pruebas con Arquitecturas Más Avanzadas

Se recomienda explorar:

- Transfer Learning: Utilizar MobileNet, ResNet o EfficientNet
- Mecanismos de atención: Para enfocar en regiones relevantes
- Modelos híbridos: Combinar CNN con RNN o transformers

7.3. Ajuste Fino de Hiperparámetros

Aplicar técnicas como Grid Search o algoritmos bayesianos para ajustar:

- Tasa de aprendizaje
- Número de capas convolucionales
- Valores de dropout rate y batch size

7.4. Integración con Aplicaciones Prácticas

- Desarrollar interfaces de usuario en tiempo real
- Incorporar detección de manos con MediaPipe
- Crear herramientas educativas para enseñar ASL

7.5. Evaluación de Impacto Social y Accesibilidad

Investigar el impacto del sistema en comunidades sordas, asegurando:

- Respeto a la diversidad cultural del ASL
- Evitar sesgos algorítmicos
- Tecnología accesible para diferentes niveles de alfabetización digital

Conclusión final: Este trabajo establece una base sólida para el reconocimiento de gestos ASL mediante CNN, pero requiere ampliaciones y refinamientos para alcanzar su potencial completo en aplicaciones accesibles y reales.

8. Referencias

Google. (s.f.). Colaboratory. Recuperado el 5 de mayo de 2025, de <https://colab.research.google.com/drive/1cEtG3fRZVKbUE4k-sBf3EZijoKpda5RH?authuser=0#scrollTo=75hJPU4z7Mkk>

Google Developers. (2024). Machine Learning Crash Course. Recuperado el 5 de mayo de 2025, de <https://developers.google.com/machine-learning/crash-course?hl=es-419>

Kaggle. (s.f.). Kaggle: Your Home for Data Science. Recuperado el 5 de mayo de 2025, de <https://www.kaggle.com/>

Keras Team. (s.f.). Keras: Deep learning API for Python, designed for human developers. Recuperado el 5 de mayo de 2025, de <https://keras.io/>

<https://docs.google.com/document/d/1LwBBDJOBEEokpBOE6TWHNNk6f6lPNl8DrhasNVMkkh>